ByteGuard: RAG Parameter Optimization and CPU-GPU Benchmarking for National Privacy Commission Complaint Procedure Questions

Balneg, Maria Cristina B.
Department of Computer Studies and Engineering
Jose Rizal University
Mandaluyong, Philippines
mariacristina.balneg@my.jru.edu

Sayat, Lyona Yvonne A.
Department of Computer Studies and Engineering
Jose Rizal University
Mandaluyong, Philippines
lyonnayvonne.sayat@jru.edu

Bartolome, Beatrice Matilda D.
Department of Computer Studies and Engineering
Jose Rizal University
Mandaluyong, Philippines
beatricematilda.bartolome@my.jru.edu

***Abstract*—This study evaluates chunk size, chunk overlap, and retrieval top-k for a retrieval-augmented generation (RAG) chatbot addressing National Privacy Commission (NPC) complaint procedures. Five configurations were benchmarked on a Colab CPU; three were repeated on a GPU. Each configuration involved three indexing measurements and three response trials per question following warm-up. The same 35-page source, five questions, and language/embedding models were maintained. For A–C, GPU average response times were 0.768–0.887 s versus 154.013–280.159 s on CPU, corresponding to 200.5–316.0 times faster responses. Some saved answers were inaccurate or incomplete, so the final configuration recommendation remains conditional on source-grounded evaluation.**

Keywords*—*RAG; chunk size; overlap; top-k; Colab; CPU–GPU benchmark; NPC complaint procedures.

I. Introduction

Retrieval-augmented generation combines relevant document passages with language-model generation. Chunk size, overlap, number of retrieved passages, and hardware affect retrieval behavior and response time. This exercise uses a prototype chatbot about Philippine NPC complaint-filing procedures to identify a practical configuration that balances speed and reliable answers. It is an educational system, not an official complaint service or a substitute for NPC guidance.

II. Experimental Methodology

A. Dataset and RAG Pipeline

The source is the 35-page NPC Complaint Procedure PDF in the ByteGuard experimental data archive [1]. It describes eligibility, form requirements, written notification, filing channels, fees, administrative processing, and supporting NPC references. The notebook splits PDF text by characters, embeds passages with sentence-transformers/all-MiniLM-L6-v2, indexes them in Chroma, retrieves the top-k chunks, and generates answers using Qwen/Qwen2.5-1.5B-Instruct [4]. The recorded executed model is Qwen; the notebook filename by itself does not indicate that GPT-OSS-20B was used.

Generation settings were max_new_tokens=256, temperature=0.2, and do_sample=True (probabilistic sampling). Five fixed questions covered written complaint requirements (Q1), the 15-calendar-day written-notice period (Q2), Rule III filing modes (Q3), the one-hour Charter service estimate (Q4), and an exact fee not established in the dataset (Q5). The CPU and GPU records have matching dataset and question-set SHA-256 values [1].

TABLE I. TESTED RAG CONFIGURATIONS

Run	Size	Overlap	Top-k	Devices
A	400	50	3	CPU/GPU
B	800	100	3	CPU/GPU
C	1,200	150	3	CPU/GPU
D	800	200	3	CPU
E	800	100	5	CPU

B. Benchmark Procedure

Configurations A–E were measured on the CPU; A–C, representing small, medium, and large chunk sizes, were also measured on a GPU. The hardware file identifies an NVIDIA RTX PRO 6000 Blackwell Server Edition for the GPU run; both embedding and language models were recorded on CUDA. The CPU file records both models on CPU. Three indexing trials per configuration measured chunking and embedding/storage separately. A warm-up preceded three answer trials for each of the five questions. Model loading was excluded from timed responses. There were 15 CPU and 9 GPU indexing measurements and 75 CPU and 45 GPU response measurements. Both error files contain zero recorded errors [1].

III. Results and Discussion

A. Recorded Processing Times

Tables II and III summarize the measured means. Averages are calculated from raw trial CSVs. On both devices, measured retrieval time was much shorter than generation time. The GPU reduced indexing time as well as response latency, although the separate Colab sessions do not control all hardware and runtime differences

TABLE II. CPU MEAN TIMES (SECONDS)

RUN	INDEX	RETR.	GEN.	TOTAL

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

A	15.340	0.031	153.979	154.013
B	13.568	0.027	218.897	218.929
C	14.430	0.032	280.122	280.159
D	15.327	0.033	214.790	214.829
E	12.940	0.036	303.838	303.880

TABLE III. GPU MEAN TIMES (SECONDS)

RUN	INDEX	RETR.	GEN.	TOTAL
A	0.216	0.003	0.764	0.768
B	0.169	0.003	0.862	0.866
C	0.151	0.003	0.882	0.887

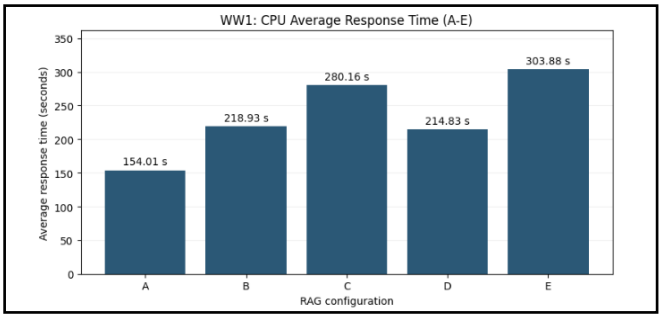


Figure 1. Average CPU response time across five RAG configurations (A-E)

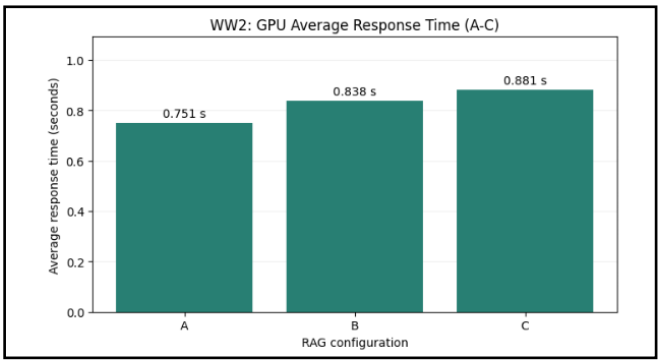


Figure 2. Average GPU response time across three RAG configurations (A–C)

B. CPU and GPU Performance Comparison

Speedup was computed as mean CPU execution time divided by mean GPU execution time for the matching configuration and metric. Values above 1 mean that the recorded GPU measurement was faster. For total response time, speedups were approximately 200.5× for A, 252.8× for B, and 316.0× for C.

TABLE IV. GPU SPEEDUP OVER CPU

RUN	INDEX	RETR.	GEN.	TOTAL
A	71.00x	9.42x	201.54x	200.49x
B	80.27x	8.41x	254.06x	252.78x
C	95.41x	9.45x	317.66x	315.97x

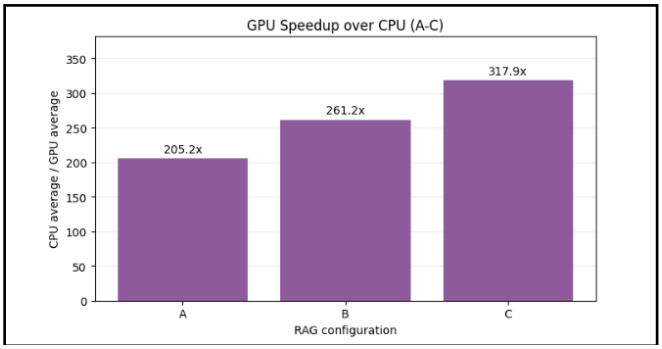


Figure 3. GPU speedup over CPU for matching RAG configurations (A–C)

C. Effects of RAG Parameters

CPU configuration A was fastest across A–E (154.01 s mean response). A–C varied both chunk size and overlap, so separate causal effects cannot be isolated. The number of chunks decreased from 275 for A to 140 for B and 100 for C, while CPU response times increased. Comparing B and D holds chunk size and top-k fixed while raising overlap from 100 to 200; mean CPU time changed from 218.93 to 214.83 s. Comparing B and E holds chunking fixed while increasing top-k from 3 to 5; mean CPU time increased to 303.88 s. With sampled generation and limited repetitions, these are descriptive observations, not definitive isolated effects.

IV. Answer Quality and Retrieval Support

The submitted archive includes full answers and retrieved passages, but not completed human ratings of correctness, retrieval relevance, or source support. Therefore, no numerical answer-quality winner is claimed. Examples indicate that GPU configuration C correctly listed Rule III filing modes in Q3, distinguished the one-hour intake estimate from case resolution in Q4, and avoided inventing an exact fee for Q5. By contrast, some A responses overextended the one-hour procedure or mislabeled PIC. Some C responses omitted complaint-certification details in Q1. These examples are illustrative rather than a complete grading of all 120 recorded answers.

Reviewers should use the dataset passages on complaint content (pp. 7–8), written notice (p. 9), filing methods (pp. 15–16), service duration (pp. 13 and 21), and fee uncertainty (pp. 4 and 11) [1]. In the Quality_Review worksheet, each response should be rated 0, 1, or 2 separately for correctness, retrieval relevance, and source support. Similarity alone does not prove factual accuracy.

V. Recommended Configuration

A provisional choice is configuration C on GPU (1,200-character chunks, 150-character overlap, top-k=3). Its mean recorded GPU response time was 0.887 s compared with 0.768 s for the faster configuration A, while inspected C answers handled several procedural questions more faithfully. GPU execution recorded no errors; however, the source archive does not contain GPU memory-utilization or cost measurements. Resource practicality remains session-dependent. The group should confirm this selection after completing full answer-quality and retrieval-relevance scoring; B also remains a candidate.

VI. Limitations and Future Work

This study uses one PDF corpus, five fixed questions, and three response trials per question. CPU and GPU sessions could differ in host environment as well as accelerator. Model load time and memory consumption were not benchmarked in a way that supports whole-session cost comparisons. Sampling was enabled, so response lengths and contents vary. Some indexing, PDF extraction, and search operations may be CPU-bound. There are no completed human quality scores or statistical confidence intervals. Future studies should independently vary chunk size and overlap, add held-out questions and negative controls, use deterministic generation or fixed seeds, log VRAM and RAM, normalize output length, and require source citations and abstention for unsupported questions.

VII. Conclusion

The recorded experiment covered five RAG configurations on CPU and three matched configurations on GPU, with common dataset and question identifiers, three measured repetitions, and zero recorded errors. GPU sessions showed much lower indexing and generation times, but sample answers demonstrate that speed cannot replace source verification. Configuration C with GPU is a provisional recommendation pending completed quality grading.

References

[1] ByteGuard, RAG Experimental Data Archive, ByteGuard_RAG_Experiment_Data.zip, Oct. 2026 (NPC_Dataset.pdf, questions, hardware, and raw measurements).

[2] National Privacy Commission, 2021 Rules of Procedure of the NPC, as Amended, 2024; cited by the dataset source register.

[3] National Privacy Commission, Citizen's Charter 2025, 1st Edition, Filing of Complaints Process; cited by the dataset source register.

[4] ByteGuard, RAG Experiment Notebook, ByteGuard_RAG_Experiment_Notebook.ipynb, Oct. 2026.

Author Contributions

Lyona Yvonne A. Sayat: Responsible for developing and implementing the Google Colab notebook, including the RAG chatbot implementation, dataset processing, and experimental setup. She conducted the CPU and GPU experiments, tested different RAG configurations, collected and analyzed the experimental results, generated performance graphs, and prepared the notebook documentation.

Maria Cristina B. Balneg: Provided additional support during the Google Colab notebook testing, particularly in checking runtime issues and assisting with troubleshooting when errors occurred.

Beatrice Matilda D. Bartolome: Responsible for preparing and writing the IEEE experimental report. She also assisted in the Google Colab notebook development and provided support during the implementation and documentation of the experiments.